

AI FARM ROBOTICS FACTORY · TRAINING · VERSION 2

COURSE: ROS 2 AMR & MANIPULATION

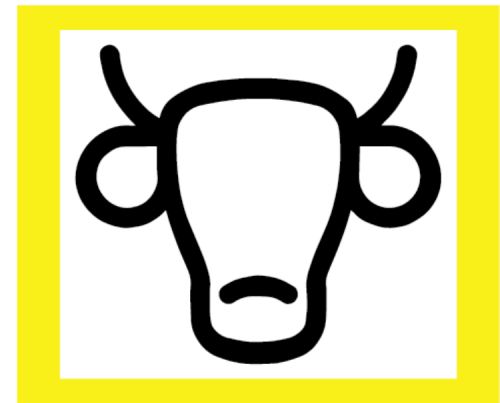
WEEK 2

Kinematic Modeling & Physics Simulation

Ubuntu 24.04 · ROS 2 Jazzy · Gazebo Harmonic · 4 hours lecture + 8 hours lab

Trailobot AMR

RB3-1200



AI FARM ROBOTICS

WEEK 2

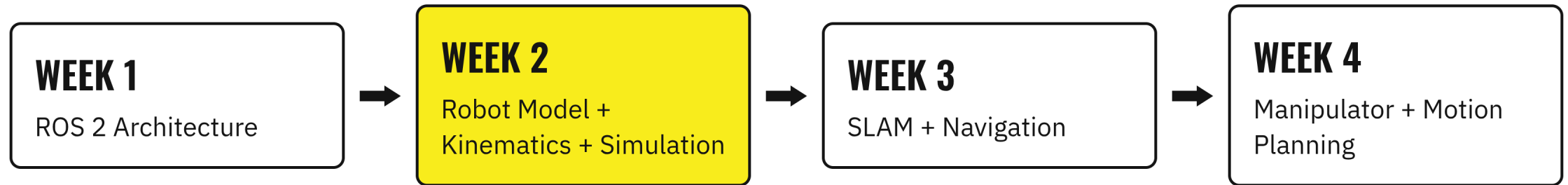
LEARNING OBJECTIVES

By the end of the lecture, students should be able to:

- 1 Explain why robots need mathematical models.
- 2 Identify links, joints, frames and DoF on Trailobot and RB3-1200.
- 3 Compute forward kinematics with frames and transformations.
- 4 Explain the difference between kinematic and physical models.
- 5 Understand the structure of URDF/Xacro.
- 6 Explain how TF2 represents robot transformations.
- 7 Spawn and check a robot model in Gazebo Harmonic.
- 8 Explain the role of physics engines in simulation.

WEEK 2 · INTRODUCTION

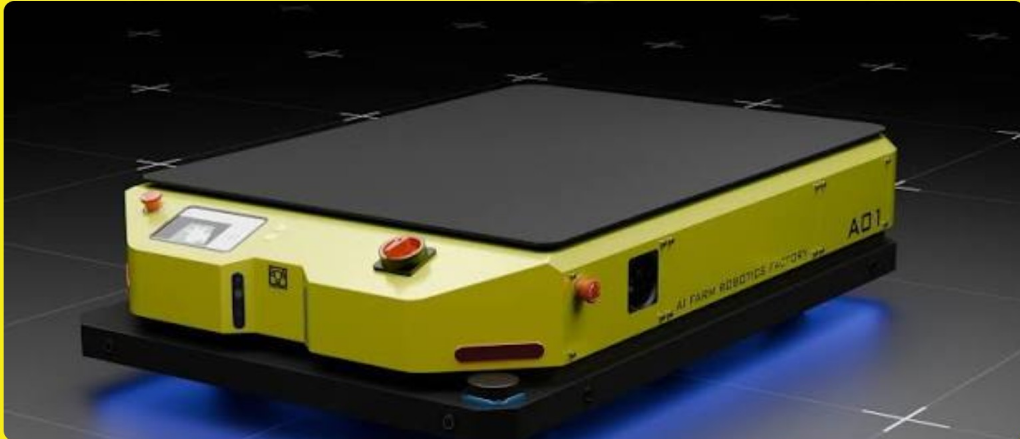
WEEK 2 INTRODUCTION

**MAIN QUESTION**

How can a computer understand the physical structure of a robot?

Today's story: Robot specification → Coordinate system → Kinematic model → URDF/Xacro → TF2 → Gazebo → Physics simulation

TWO ROBOTS THIS WEEK



TRAILOBOT AMR

Mobile robot: chassis, wheels, LiDAR, IMU

Used for frames, footprint and simulation

Photo: AI Farm Robotics Factory (A01)



RB3-1200

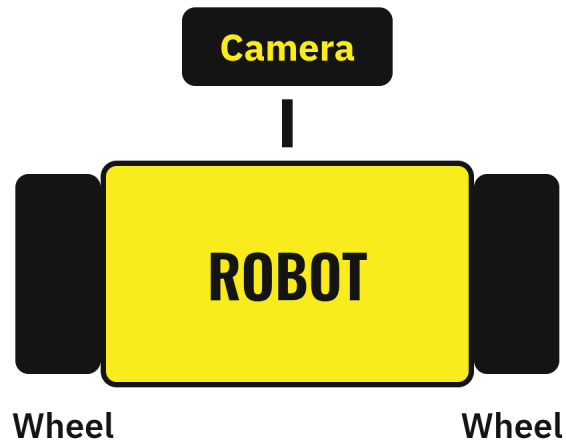
Collaborative arm: 6 joints, reach 1200 mm, payload 3 kg

Used for joints, FK, frames and URDF

Photo and specs: Rainbow Robotics

PART 1 · MODELING BASICS

WHAT MUST THE ROBOT COMPUTER KNOW?



Does the computer know where the camera is?

Does it know the wheel radius?

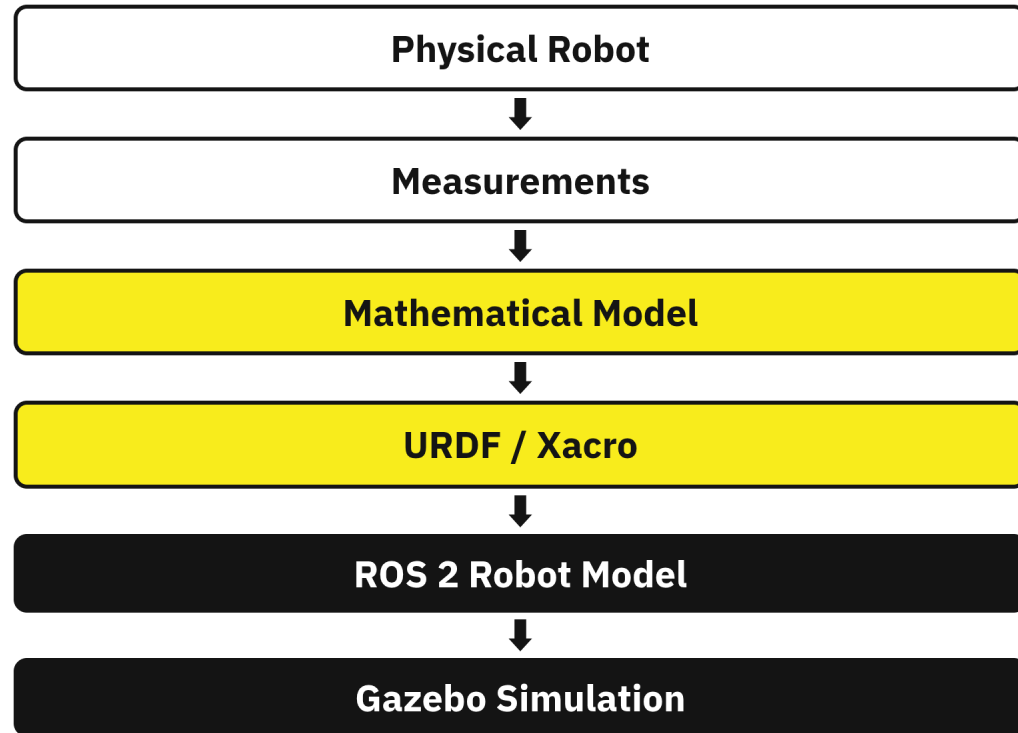
Does it know which parts can rotate?

Does it know the distance between two joints?

No — we need a robot model.

PART 1 · MODELING BASICS

PHYSICAL ROBOT TO DIGITAL ROBOT



WHAT THIS PROCESS CREATES

A digital representation of the robot.

We measure the real robot, describe it mathematically, write it in a format ROS 2 understands, and then simulate it.

TRAILOBOT AMR: STRUCTURE AND SENSORS

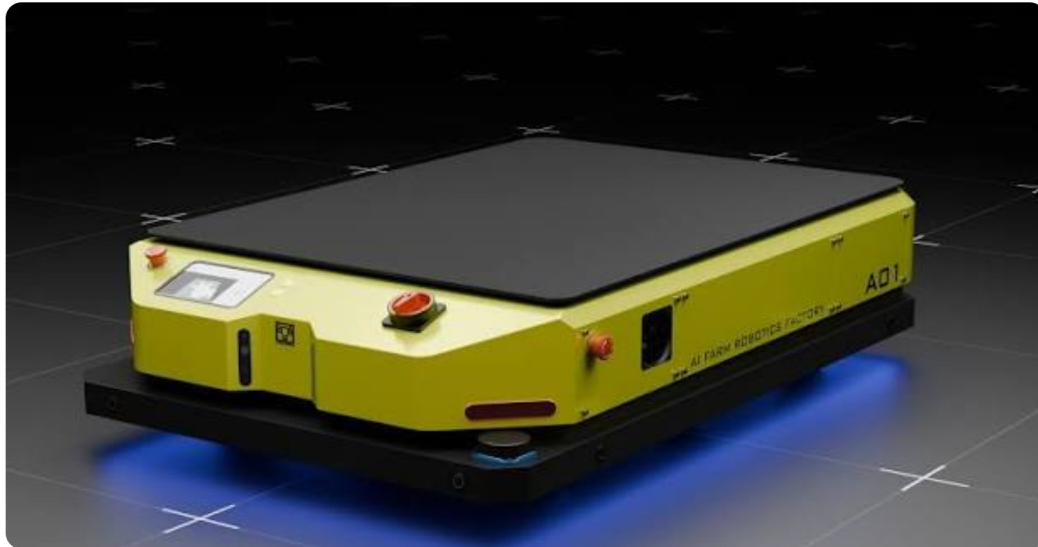


Photo: AI Farm Robotics Factory AMR (AO1). Replace with your own Trailobot photo if it differs.

REAL PART → MODEL ELEMENT

Part	Frame / joint	Used for
Chassis	base_link	Footprint
Wheels	wheel joints	Odometry
LiDAR	lidar_link	SLAM scans
IMU	imu_link	Orientation
Camera	camera_link	Vision

Typical mapping. Confirm wheel layout and sensor positions on the real robot.

A correct robot description is the base for SLAM, Nav2 and obstacle avoidance.

PART 1 · MODELING BASICS

ROBOT SPECIFICATION

Before creating a model, we need to define:

Length	Width	Height
Mass	Wheel radius	Wheel position
Joint position	Joint limits	Sensor position

EXAMPLE ROBOT

L = 500 mm

W = 400 mm

H = 200 mm

Mass = 25 kg

Wheel radius = 75 mm

PART 1 · MODELING BASICS

WHY ACCURATE SPECIFICATION MATTERS

REAL ROBOT

$$R = 0.075 \text{ m}$$

WRONG MODEL

$$R = 0.100 \text{ m}$$

Command velocity



Wheel rotation



Robot movement

 **Incorrect motion**

Same wheel speed, ground speed $v = \omega \times R$. The wrong radius makes the simulated robot move **33% farther** than the real one ($0.100 / 0.075 \approx 1.33$).

Good simulation starts with a **good robot specification.**

PART 1 · MODELING BASICS

WHAT IS A LINK?

A **link** represents a rigid part of a robot.

EXAMPLES

base_link

chassis

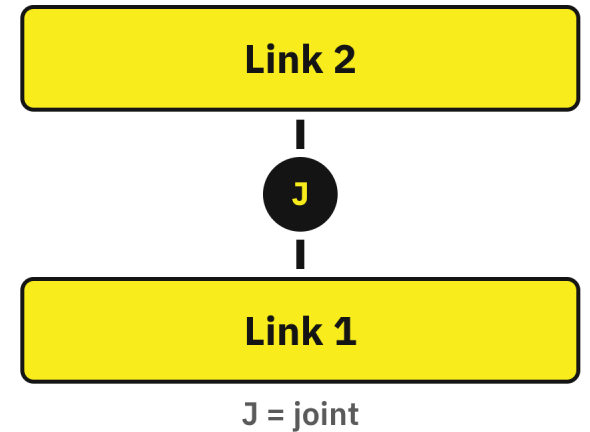
left_wheel

right_wheel

arm_link_1

arm_link_2

gripper



WHAT IS A JOINT?

A **joint** defines how two links can move relative to each other.

FIXED

No relative motion. Link
— Link.

REVOLUTE

Rotates around an axis
within limits.

CONTINUOUS

Rotates without limits, like
a wheel.

PRISMATIC

Slides along an axis
within limits.

In practice: robot wheels use continuous joints; arm joints usually use revolute joints; a linear lift uses a prismatic joint.

DEGREES OF FREEDOM

Degree of Freedom (DoF) = **independent motion variable.**

Fixed joint

0 DoF

Revolute joint

1 rotational DoF

Prismatic joint

1 translational DoF

OUR ROBOTS

RB3-1200: J1 ... J6 → 6 DoF

Trailobot base: x, y, yaw

ARM = 6 DOF



PART 1 · MODELING BASICS

WHY COORDINATE FRAMES?

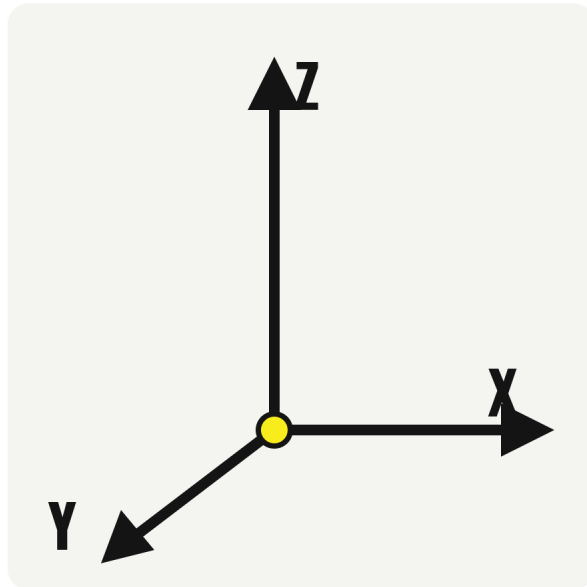
Where is the LiDAR?

Relative to what?

Every position must be described relative to a coordinate frame.

PART 1 · MODELING BASICS

COORDINATE SYSTEM



ROS CONVENTION (RIGHT-HANDED)

X Forward**Y** Left**Z** Up

Use the same convention consistently in every model.

PART 1 · MODELING BASICS

JOINT POSITION VS END-EFFECTOR POSE

JOINT SPACE

$$q = (\theta_1 \dots \theta_6)$$

What the motors and encoders report

Forward kinematics



TASK SPACE

$$\text{pose} = (x, y, z, r, p, y)$$

Where the tool is and how it is turned

RB3-1200

Six joint angles → tool flange pose in base_link

TRAILOBOT

Wheel rotations → base_link pose (x, y, yaw) in odom

Several joint sets can give the same pose. Finding them is **inverse kinematics** (Week 4).

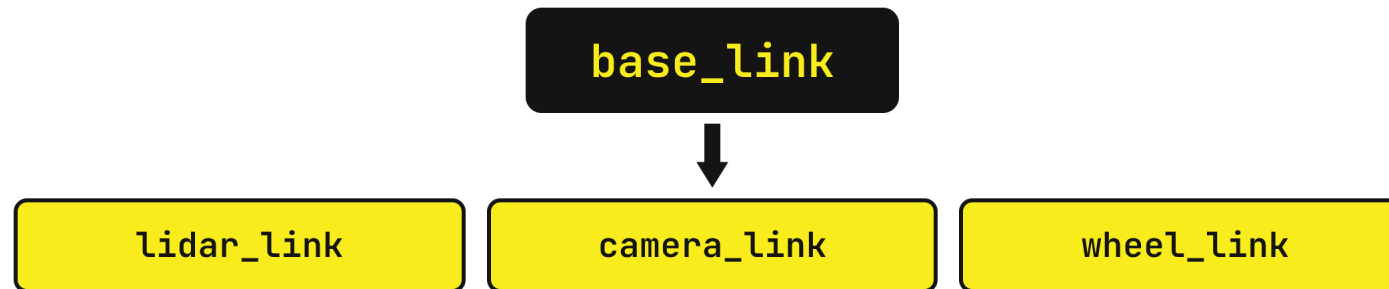
PART 1 · MODELING BASICS

MOBILE ROBOT VS MANIPULATOR

	Trailobot (mobile)	RB3-1200 (arm)
Base	Moves through the world	Fixed to the workcell
Joints	Wheels: continuous	Revolute, with limits
State	Pose x, y, yaw	Joint angles q1 ... q6
Frame chain	map → odom → base_link	base_link → ... → tool0
Model is used for	Footprint, odometry, Nav2	Reach, collisions, planning

Both are trees of links and joints. Only the joints and the main questions differ.

ROBOT COORDINATE FRAMES

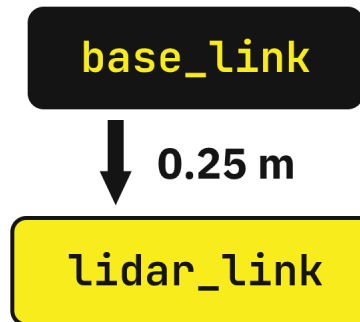


Each component has its own frame.

The robot body, every sensor and every wheel get a frame, so each position can be described relative to something meaningful.

This prepares students for TF2.

WHAT IS A TRANSFORMATION?



Where is lidar_link relative to base_link?

A TRANSFORMATION CONTAINS

TRANSLATION

how far the frame is shifted

+

ROTATION

how the frame is turned

TRANSLATION

$$\mathbf{p} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

EXAMPLE: LASER_LINK

$x = 0.20$ m

$y = 0.00$ m

$z = 0.15$ m

The LiDAR is 20 cm forward and 15 cm above the reference frame.

ROTATION

ROLL

Rotation around X

PITCH

Rotation around Y

YAW

Rotation around Z

FOR A MOBILE ROBOT

Yaw is especially important because the robot rotates on the ground plane.

HOMOGENEOUS TRANSFORMATION MATRIX

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}$$

R = rotation

p = translation

THE IMPORTANT IDEA

A transformation tells us where one coordinate frame is relative to another.

WHAT IS KINEMATICS?

Kinematics describes robot motion without considering forces and torques.

KINEMATICS

Position

Velocity

Acceleration

“How does the robot move?”

DYNAMICS

Mass

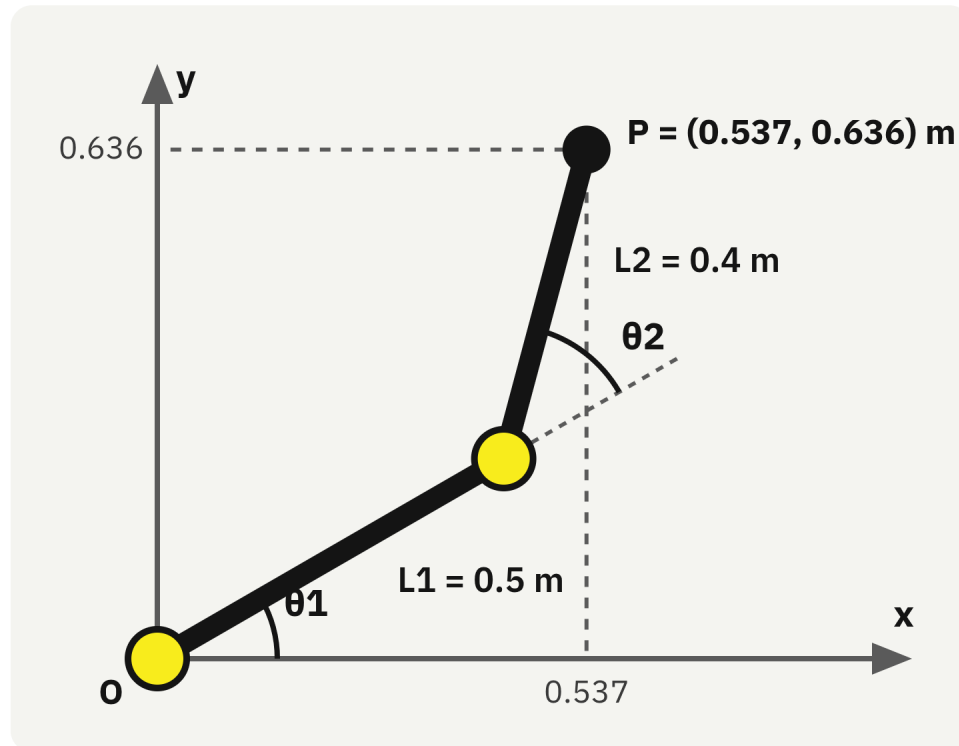
Force

Torque

“Why does the robot move?”

This distinction is important before Gazebo.

WORKED EXAMPLE: TWO-LINK ARM



STEP 1: WRITE THE EQUATIONS

$$x = L1 \cdot \cos \theta1 + L2 \cdot \cos(\theta1 + \theta2)$$

$$y = L1 \cdot \sin \theta1 + L2 \cdot \sin(\theta1 + \theta2)$$

STEP 2: INSERT THE NUMBERS

$$L1 = 0.5 \text{ m}, L2 = 0.4 \text{ m}, \theta1 = 30^\circ, \theta2 = 45^\circ$$

$$x = 0.5 \cdot \cos 30^\circ + 0.4 \cdot \cos 75^\circ$$

$$= 0.433 + 0.104 = 0.537 \text{ m}$$

$$y = 0.5 \cdot \sin 30^\circ + 0.4 \cdot \sin 75^\circ$$

$$= 0.250 + 0.386 = 0.636 \text{ m}$$

$$\text{tool angle } \psi = \theta1 + \theta2 = 75^\circ$$

PART 2 · KINEMATICS AND TF2

FORWARD KINEMATICS AS A CHAIN

ONE LINK (2D)

$\cos \theta$	$-\sin \theta$	$L \cdot \cos \theta$
$\sin \theta$	$\cos \theta$	$L \cdot \sin \theta$
0	0	1

$$T(\theta, L) = \text{Rot}(\theta) \cdot \text{Trans}(L, \theta)$$

$$T = T1 \cdot T2 (30^\circ, 45^\circ)$$

0.259	-0.966	0.537
0.966	0.259	0.636
0	0	1

Last column = tip position (m)

1 · Build T1



2 · Build T2



3 · Multiply



4 · Read x, y

RB3-1200: six transforms, one per joint. TF2 multiplies the chain for us.

FORWARD KINEMATICS



We know the joint positions and calculate where the end-effector is.

EXAMPLE: 3-DOF ARM (BASE, SHOULDER, ELBOW)

θ1 base: rotates about the vertical Z axis

θ2 shoulder: lifts the whole arm

θ3 elbow: bends the forearm

$$r = L2 \cdot \cos(\theta_2) + L3 \cdot \cos(\theta_2 + \theta_3)$$

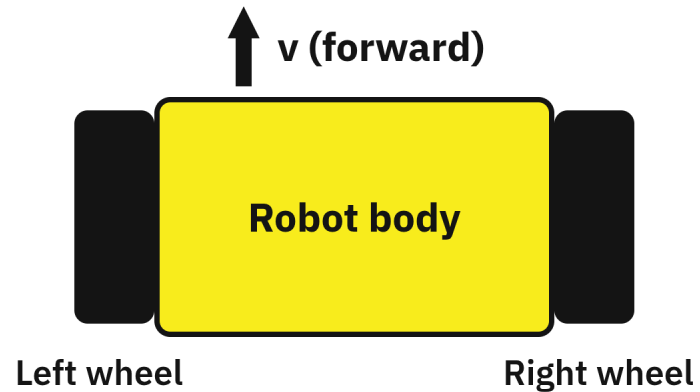
$$x = r \cdot \cos(\theta_1)$$

$$y = r \cdot \sin(\theta_1)$$

$$z = d1 + L2 \cdot \sin(\theta_2) + L3 \cdot \sin(\theta_2 + \theta_3)$$

$$d1 = \text{base height} \cdot L2 = \text{upper arm} \cdot L3 = \text{forearm}$$

MOBILE ROBOT KINEMATICS



Wheel velocities determine the robot's linear and angular motion.

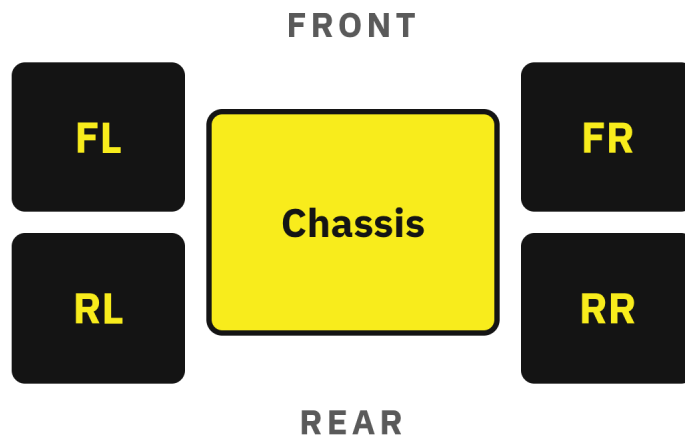
$$v = (v_R + v_L) / 2$$

$$\omega = (v_R - v_L) / L$$

v_R, v_L = right and left wheel speeds · L = distance between the wheels

A useful bridge to the Nav2 week: Nav2 sends v and ω , the robot turns them into wheel speeds.

MECANUM ROBOT EXAMPLE



THREE PLANAR MOTIONS

v_x Forward / backward

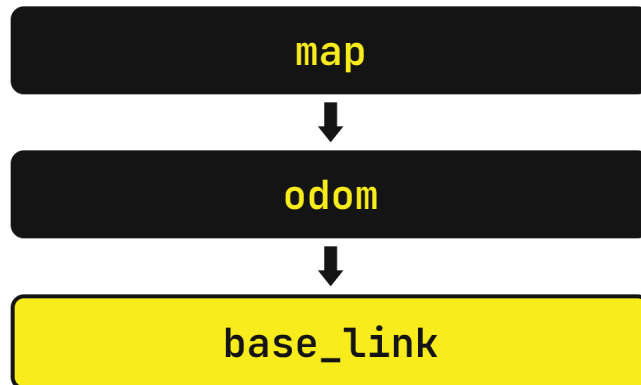
v_y Sideways

ω_z Rotation

Unlike a differential-drive robot, a mecanum robot can move sideways without turning.

WHAT IS TF2?

TF2 manages relationships between coordinate frames over time.

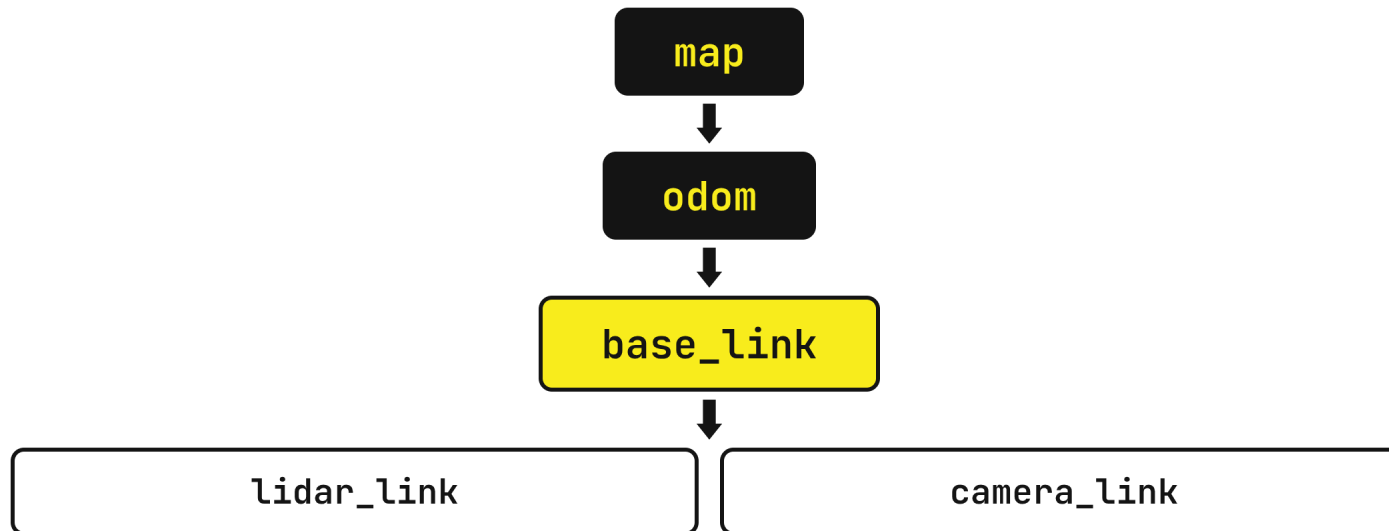


map: fixed world frame

odom: smooth, continuous odometry frame

base_link: frame attached to the robot body

TRANSFORM TREE



Every sensor and robot component needs a meaningful frame relationship.

Each arrow is a transform: the child frame is positioned relative to its parent. Chain them to find any frame in the map.

FRAME TREES OF TWO ROBOTS

TRAILOBOT

```
map
└─ odom
   └─ base_link
      ├── left_wheel
      ├── right_wheel
      ├── lidar_link
      ├── imu_link
      └─ camera_link
```

Moving base: odom → base_link changes as it drives.

RB3-1200

```
base_link
└─ link_1
   └─ link_2
      └─ link_3
         └─ link_4
            └─ link_5
               └─ link_6
                  └─ tool0
```

Fixed base: six joints move the tool (tool0).

Frame names are illustrative. Real names come from the URDF you load.

RB3-1200: JOINTS AND FRAMES



Image: Rainbow Robotics RB3-1200

Reach 1200 mm

Payload 3 kg

Repeatability ± 0.05 mm

Weight 22.4 kg

Frames: base_link $\rightarrow$ link_1 ... link_6 $\rightarrow$ tool0

JOINT RANGES AND SPEEDS (RAINBOW ROBOTICS)

Joint	Range	Max speed
J1	$\pm 360^\circ$	$\pm 180^\circ/\text{s}$
J2	$\pm 360^\circ$	$\pm 180^\circ/\text{s}$
J3	$\pm 165^\circ$	$\pm 180^\circ/\text{s}$
J4	$\pm 360^\circ$	$\pm 180^\circ/\text{s}$
J5	$\pm 360^\circ$	$\pm 180^\circ/\text{s}$
J6	$\pm 360^\circ$	$\pm 180^\circ/\text{s}$

Six revolute joints = 6 DoF. Write limits for **every** joint in the URDF.

INSPECT TRANSFORMS IN ROS 2

```
ros2 run tf2_tools view_frames
```

Saves a PDF of the whole frame tree

```
ros2 run tf2_ros tf2_echo base_link lidar_link
```

Prints one transform: translation and rotation

```
ros2 topic echo /tf_static --once
```

Fixed transforms: sensors and fixed joints

```
ros2 topic echo /tf --once
```

Moving transforms: wheels, arm joints, odom

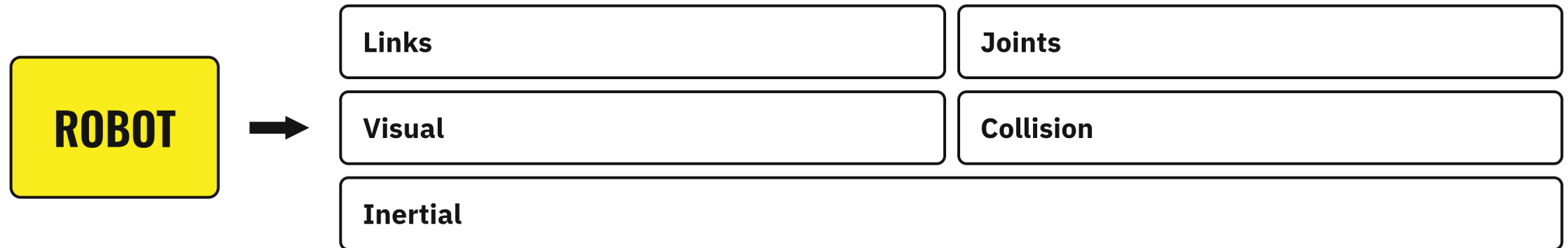
```
rviz2
```

Add the TF display and set the Fixed Frame

Try it on Trailobot first, then on the RB3-1200 model in RViz.

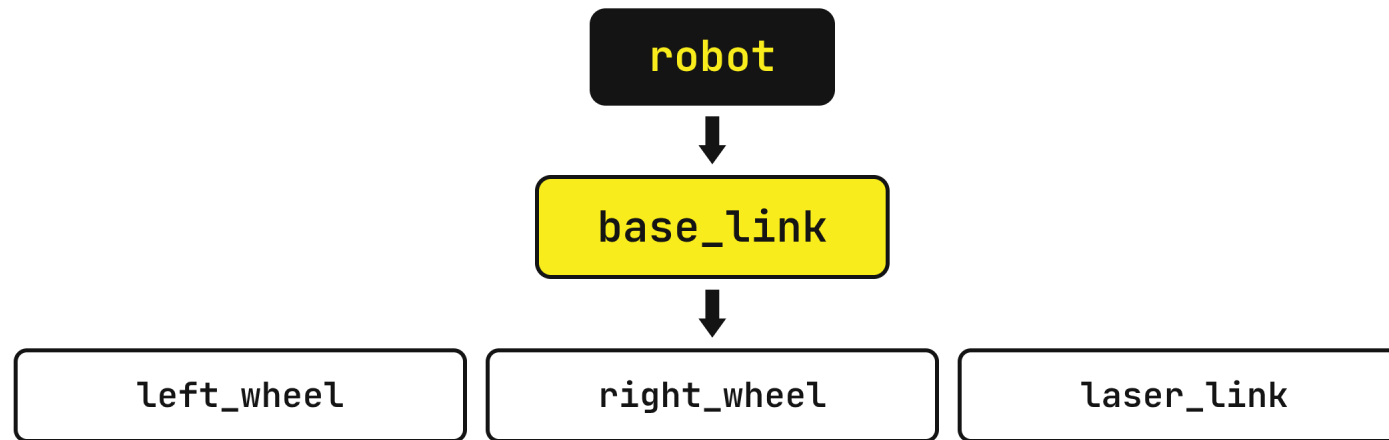
WHAT IS URDF?

URDF (Unified Robot Description Format) describes the structure of a robot.



A URDF is an XML file. ROS 2 tools such as RViz, robot_state_publisher and Gazebo all read it.

URDF ROBOT STRUCTURE



URDF creates a tree-like description of the robot.

Every link has one parent joint, except the root link (base_link). Each wheel and sensor is attached to base_link by a joint.

LINK STRUCTURE

```
<link name="base_link">
  <visual>
    ...
  </visual>
  <collision>
    ...
  </collision>
  <inertial>
    ...
  </inertial>
</link>
```

Visual

what we see

Collision

what interacts physically

Inertial

mass and inertia

JOINT STRUCTURE

```
<joint name="left_wheel_joint"  
  type="continuous">  
  <parent link="base_link"/>  
  <child link="left_wheel"/>  
  <origin .../>  
  <axis xyz="0 1 0"/>  
</joint>
```

IDENTIFY

type

parent

child

origin

axis

This continuous joint lets the wheel spin about the Y axis relative to base_link.

JOINT ORIGINS, AXES AND LIMITS

```
<joint name="joint_3" type="revolute">  
  <parent link="link_2"/>  
  <child link="link_3"/>  
  <origin xyz="0 0 0.4" rpy="0 0 0"/>  
  <axis xyz="0 1 0"/>  
  <limit lower="-2.88" upper="2.88"  
    effort="..." velocity="3.14"/>  
</joint>
```

Limits from the RB3-1200 J3 datasheet. Origin and axis are illustrative.

ORIGIN

Child frame pose in the parent: xyz (m),
rpy (rad)

AXIS

Rotation or sliding direction, in the
child frame

LIMIT

lower/upper (rad), effort (N·m),
velocity (rad/s)

RB3-1200 J3: $\pm 165^\circ = \pm 2.88 \text{ rad}$ · speed $180^\circ/\text{s} = 3.14 \text{ rad/s}$

WHY XACRO?

PROBLEM

Large URDF files become repetitive:
1000+ lines.



SOLUTION

Xacro = XML macros

BENEFITS

Variables

Reusable components

Math expressions

Modular descriptions

Example: define wheel_radius once as a variable, then reuse it for both wheels instead of copy-pasting the same block twice.

URDF VS XACRO

URDF	Xacro
Robot description	Robot description generator
XML	XML + macros
More repetitive	Reusable
Simple robots	Larger / complex robots



Important: Xacro is processed to generate URDF.

URDF VS TF2

URDF

Describes the robot structure.

TF2

Maintains frame relationships over time.

URDF



`robot_state_publisher`



`/tf · /tf_static`

Connect to Week 1: ROS 2 nodes communicate transformations through topics such as `/tf` and `/tf_static`.

PART 3 · URDF AND XACRO

VALIDATE A ROBOT MODEL

```
xacro robot.urdf.xacro > robot.urdf
```

1 · Expand Xacro into plain URDF

```
check_urdf robot.urdf
```

2 · Parse it and print the link tree

```
urdf_to_graphviz robot.urdf
```

3 · Draw the tree as a PDF

```
ros2 run tf2_tools view_frames
```

4 · Compare the live TF tree with your plan (RViz2 too)

```
gz sdf -k model.sdf
```

5 · Check an SDF file for Gazebo Harmonic

CHECKLIST

Units: metres, kilograms, radians

One root link, no loops

Axis and limits on every moving joint

Mass above 0 and plausible inertia

Mesh scale and collision shapes

WHY SIMULATION?

Why not test everything directly on a real robot?

Hardware cost

Safety

Time

Hardware damage

Reproducibility

SIMULATION ALLOWS

Test



Fail



Modify



Test again

WHAT IS GAZEBO HARMONIC?

Gazebo is a robotics simulation environment.

IT CAN SIMULATE

Robot models

Sensors

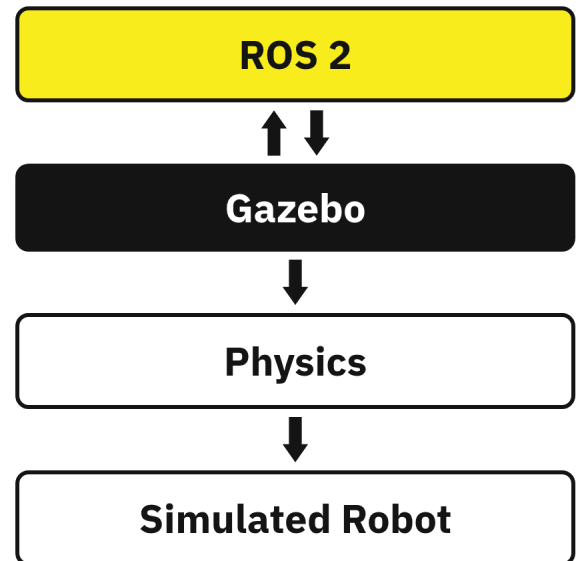
Environment

Gravity

Collision

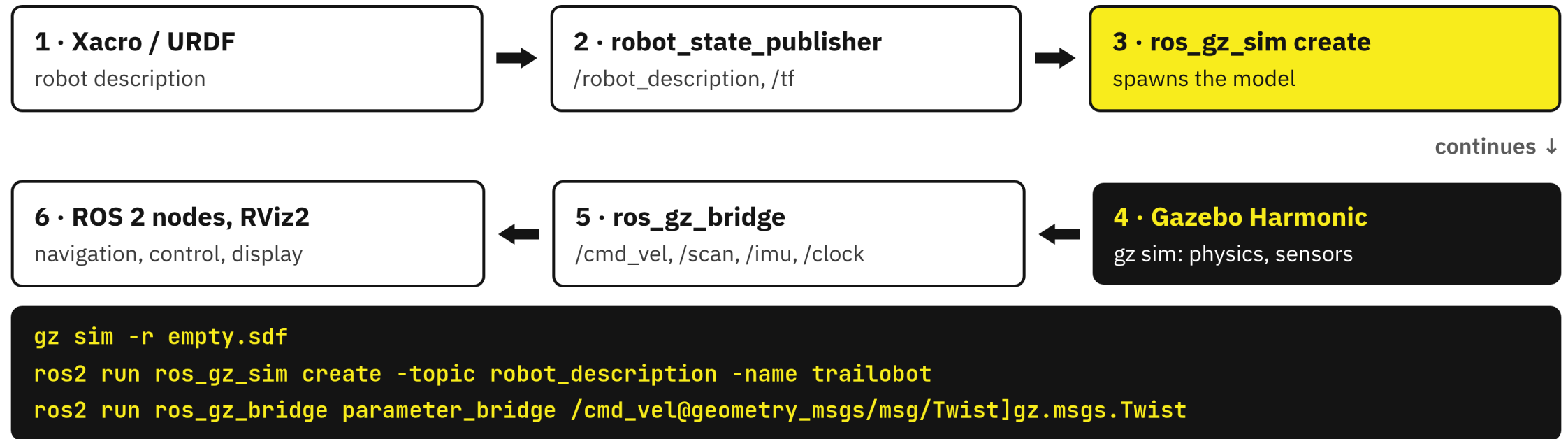
Physics

Robot motion



PART 4 · GAZEBO HARMONIC

GAZEBO HARMONIC WITH ROS 2 JAZZY



Use: gz sim and the ros_gz packages. **Not:** Gazebo Classic (gazebo, gazebo_ros_pkgs).

KINEMATIC VS PHYSICAL SIMULATION

KINEMATIC MODEL

Position

Orientation

Joint movement

PHYSICAL MODEL

Mass Force

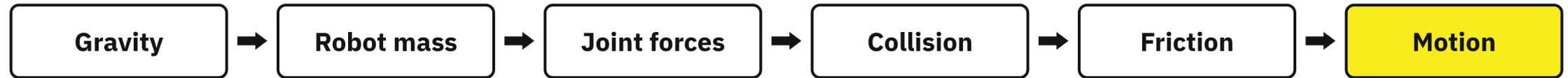
Gravity Torque

Friction Velocity

Collision Acceleration

Therefore: **URDF structure alone is not enough** for realistic physics simulation.

WHAT DOES A PHYSICS ENGINE DO?



Physics simulation calculates how objects behave **under physical constraints**.

The engine repeats this calculation in very small time steps, so the simulated robot falls, rolls, collides and slides like a real one.

COLLISION MODEL

VISUAL MODEL

What humans see

Detailed robot mesh

COLLISION MODEL

What physics sees

Simple collision geometry

WHY SIMPLIFIED COLLISION GEOMETRY?

Boxes, cylinders and spheres are much faster to compute than detailed meshes, and give more stable contacts.

INERTIA & MASS

$$F = m \cdot a$$

Force = mass × acceleration

$$\tau = I \cdot \alpha$$

Torque = moment of inertia × angular acceleration

Mass

how easily the robot accelerates

Inertia

how easily the robot rotates

Students don't need a deep dynamics course here: mass and inertia affect how the simulated robot accelerates and rotates.

TRAILOBOT IN SIMULATION

WHEEL CONTACT

Cylinder collision on each wheel. Friction sets grip: too low and the wheels spin, too high and the robot jerks.

MASS AND INERTIA

Example: 25 kg box, $0.5 \times 0.4 \times 0.2$ m. I_{xx} 0.417, I_{yy} 0.604, I_{zz} 0.854 $\text{kg}\cdot\text{m}^2$.

FOOTPRINT

A simple box or polygon for collision and for the Nav2 footprint. It must cover the real chassis.

FRAMES AND CLOCK

lidar_link and imu_link at measured offsets. Set use_sim_time to true so TF and Gazebo share one clock.

Why it matters: a wrong footprint gives paths that do not fit; a wrong wheel radius or wheel separation gives odometry drift (Week 3).

ARMS IN SIMULATION

JOINT LIMITS

Copy the datasheet: RB3-1200 J3 $\pm 165^\circ$, the others $\pm 360^\circ$.

COLLISION

Simple convex shapes. Ignore contact between neighbouring links.

CONTROL

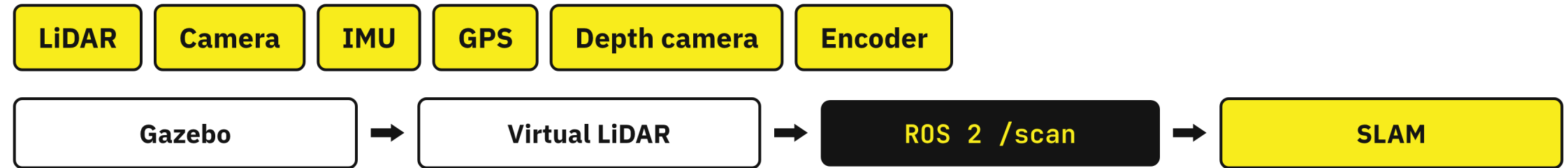
ros2_control with gz_ros2_control moves the joints (Week 4).

	Educational model	Manufacturer-accurate model
Geometry	Boxes and cylinders	Vendor CAD meshes
Limits	Rounded values	Exact datasheet values
Inertia	Estimated from shapes	From CAD or measured
Source	Written by students	Vendor or community: check licence

PART 4 · GAZEBO HARMONIC

SIMULATED SENSORS

A simulation can generate:



This directly prepares students for **Week 3**.

PART 4 · GAZEBO HARMONIC

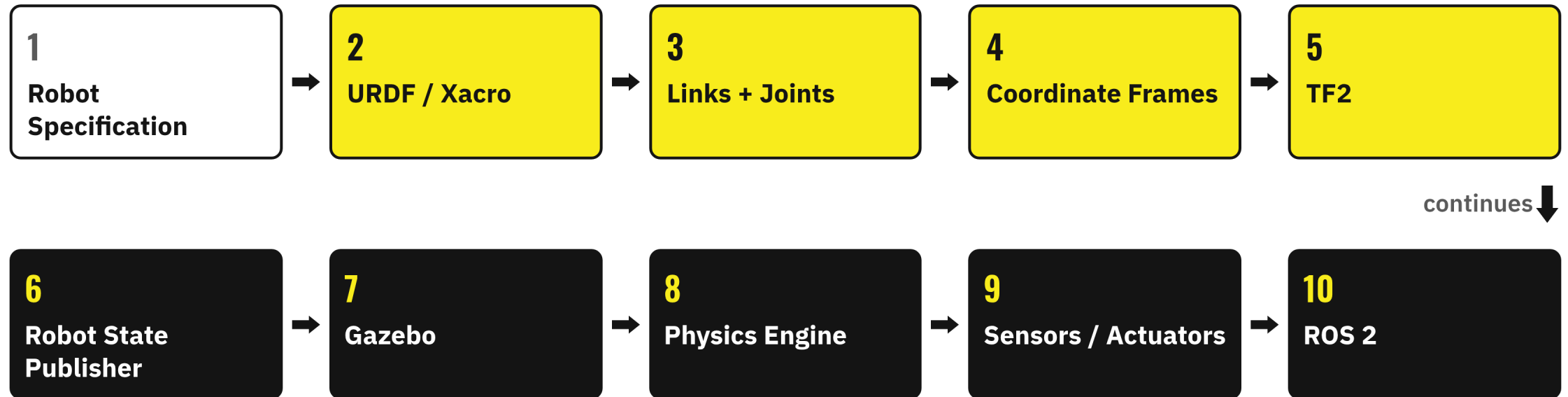
COMMON MODELING AND SIMULATION ERRORS

Symptom	Likely cause	First check
Falls through the floor	No collision geometry	collision on each link
Jitters or explodes	Bad inertia or overlap	inertial and self-contact
Wheels spin, robot stays	Low friction or wrong axis	friction and axis xyz
Tiny or huge model	mm instead of m	units and mesh scale
Does not move	Bridge or topic missing	ros_gz_bridge, gz topic -l
RViz: no transform	Missing frame or clock	view_frames, use_sim_time

Change one thing at a time and re-run the validation steps.

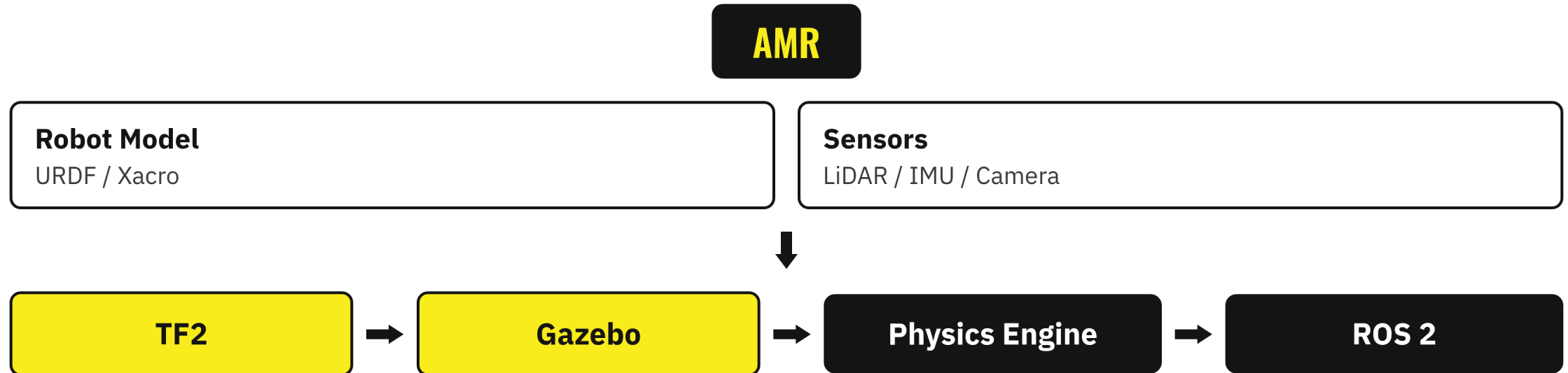
INTEGRATION

FROM ROBOT MODEL TO SIMULATION



INTEGRATION

COMPLETE AMR EXAMPLE



INTEGRATION

LIVE DEMO

Build and simulate a simple mobile robot

- | | | | | |
|----------------------------|---------------------------|------------------------------|------------------------|------------------------------|
| 1
Create package | 2
Create Xacro | 3
Define base_link | 4
Add wheels | 5
Add LiDAR , IMU |
| 6
Publish TF | 7
Launch Gazebo | 8
Visualize robot | 9
Move robot | 10
Inspect TF tree |

Steps 1-5 build the model; steps 6-10 bring it to life. Keep the commands in your teaching notes, not on the slide.

WRAP-UP

KEY TAKEAWAYS

- 1 A robot model = links, joints, frames and physical properties.
- 2 DoF = independent motion variables. The RB3-1200 has 6.
- 3 Forward kinematics chains transforms from joints to pose.
- 4 TF2 holds the frame tree. URDF feeds it through robot_state_publisher.
- 5 URDF has visual, collision and inertial parts. Xacro avoids repetition.
- 6 Gazebo Harmonic needs mass, inertia, collision and friction.

Good model → good simulation → good SLAM, Nav2 and manipulation.

WRAP-UP

KNOWLEDGE CHECK

- 1 What is a link?
- 2 What is a joint?
- 3 What is a DoF?
- 4 Why do we need coordinate frames?
- 5 What is a transformation?
- 6 What is the difference between URDF and Xacro?
- 7 What is TF2?
- 8 What is the difference between visual and collision geometry?
- 9 What does a physics engine calculate?
- 10 What is the difference between kinematics and dynamics?

WRAP-UP

PREPARING FOR THE WEEK 2 LAB

CHECK BEFORE THE LAB

```
gz sim --version
```

```
ros2 pkg list | grep ros_gz
```

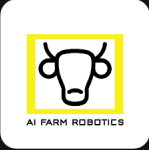
```
which check_urdf xacro
```

Ubuntu 24.04, ROS 2 Jazzy, Gazebo Harmonic.

LAB TASKS (8 HOURS)

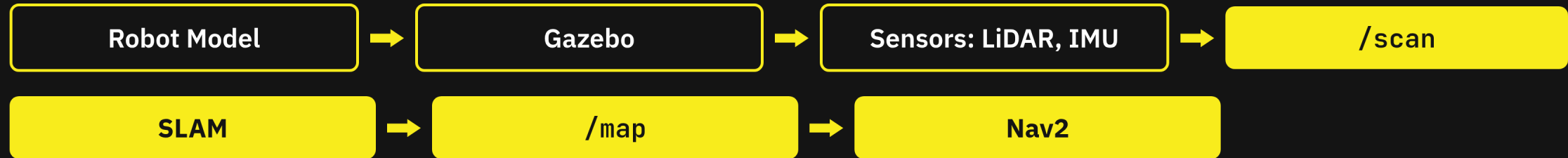
- 1 Measure Trailobot, fill the specification table
- 2 Write base_link, wheels, LiDAR and IMU in Xacro
- 3 Validate: check_urdf and view_frames
- 4 Spawn in Gazebo Harmonic, drive with /cmd_vel
- 5 Inspect TF and fix a broken model
- 6 Extension: view an RB3-1200 model in RViz2

Hand in: the package, a TF tree PDF and a Gazebo screenshot.



PART 16 · NEXT · WEEK 3

FROM SIMULATION TO AUTONOMY



“Now that our robot exists in simulation and has sensors, how can it build a map and navigate autonomously?”

That becomes Week 3.

WRAP-UP

REFERENCES AND IMAGE CREDITS

MANUFACTURER DATA (CHECKED 9 OCT 2026)

RB3-1200: rainbow-robotics.com/en/products/rb3-1200/

ROS AND GAZEBO

Gazebo Harmonic, ROS 2 spawn: gazebo-sim.org/docs/harmonic/ros2_spawn_model/

ROS 2 Jazzy docs: docs.ros.org/en/jazzy/

REP 103 units, REP 105 frames: ros.org/reps/rep-0103.html, ros.org/reps/rep-0105.html

URDF XML spec: wiki.ros.org/urdf/XML

IMAGE CREDITS

Trailobot photo: AI Farm Robotics Factory (AO1), project file.

RB3-1200 photo: supplied by the course author, credit Rainbow Robotics. Check reuse terms.

STATUS

Specifications checked against the manufacturer pages. Commands and simulation behaviour not yet run on a real robot or in Gazebo.